

HW_04

Henry Traynor

Due: 5:00pm, 15 September

Create a self-contained R script or Rmarkdown/Quarto file to do the following:

Using the simulated data, compare the prior and posterior predictive distributions of λ for each of the 3 forest types.

Hint: because forest type is categorical (not continuous), you can visualize the prior and posterior predictive distributions as histograms for each forest type.

```
set.seed(314159) # for reproducibility

nSites <- 100
nVisits <- 4

forest <- factor(sample(c("Hardwood", "Mixed", "Pine"), nSites, replace=TRUE))
forestMixed <- ifelse(forest=="Mixed", 1, 0) ## Dummy
forestPine <- ifelse(forest=="Pine", 1, 0) ## Dummy
temp <- matrix(rnorm(nSites*nVisits), nrow=nSites, ncol=nVisits)

# true coeffs
beta0 <- 0
beta1 <- -1
beta2 <- 1
lambda <- exp(beta0 + beta1*forestMixed + beta2*forestPine)
alpha0 <- -2
alpha1 <- 1
p <- plogis(alpha0 + alpha1*temp)

# true lambda plots
lambda.true <- c(
  Hardwood = exp(beta0),
  Mixed = exp(beta0 + beta1),
  Pine = exp(beta0 + beta2)
)
lambda.true

## Hardwood      Mixed      Pine
## 1.0000000 0.3678794 2.7182818

# simulate occupancy and detection data
N <- rpois(nSites, lambda=lambda) ## local abundance
y <- matrix(NA, nrow=nSites, ncol=nVisits)
for(i in 1:nSites) {
```

```

    y[i,] <- rbinom(nVisits, size=N[i], prob=p[i,])
}

```

```

# summary statistics
maxCounts <- apply(y, 1, max)
table(maxCounts)

```

```

## maxCounts
##  0  1  2  3  4
## 61 34  2  2  1

```

```

naiveOccupancy <- sum(maxCounts>0)/nSites
naiveOccupancy

```

```

## [1] 0.39

```

```

naiveAbund <- sum(maxCounts)
naiveAbund

```

```

## [1] 48

```

```

# prep data in named list
jags.data <- list(y=y,
                 forestMixed=forestMixed,
                 forestPine=forestPine,
                 nSites=nSites,
                 temp=temp,
                 nOccasions=nVisits)
# initial values for MCMC
jags.inits <- function() {
  list(beta0=rnorm(1), alpha0=rnorm(1), N=maxCounts)
}
# parameters to save
jags.pars <- c("beta0", "beta1", "beta2",
              "alpha0", "alpha1", "totalAbundance")

```

```

# run model
library(jagsUI)
jags.post.samples <- jags.basic(data=jags.data,
                                inits=jags.inits,
                                parameters.to.save=c(jags.pars, "N"),
                                model.file="Nmix-model-covs.jag",
                                n.chains=3,
                                n.adapt=100,
                                n.burnin=0,
                                n.iter=2000,
                                parallel=TRUE)

```

```

##
## Processing function input.....
##

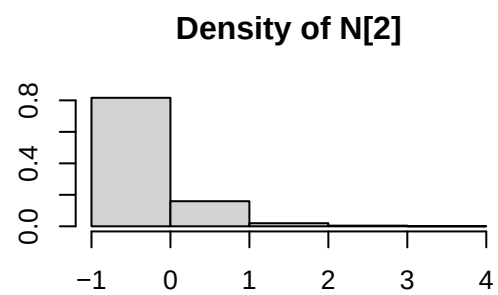
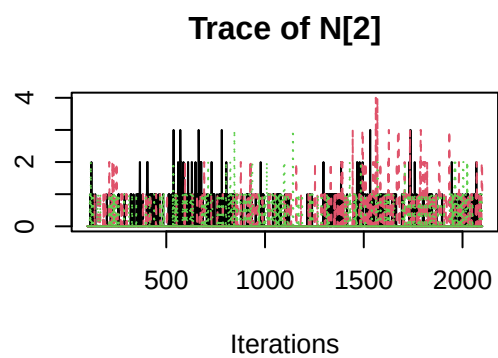
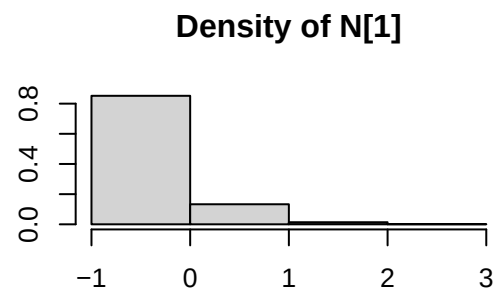
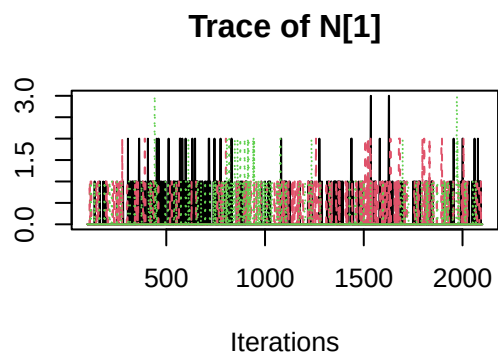
```

```
## Done.
##
## Beginning parallel processing using 3 cores. Console output will be suppressed.
##
## Parallel processing completed.
##
## MCMC took 0.101 minutes.
```

```
summary(jags.post.samples[,jags.pars])
```

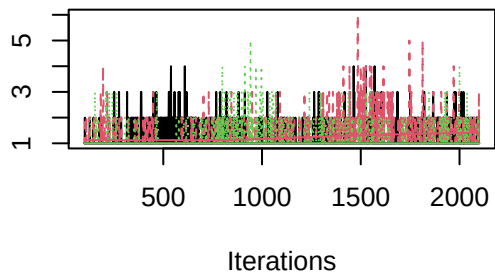
```
##
## Iterations = 101:2100
## Thinning interval = 1
## Number of chains = 3
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##              Mean      SD Naive SE Time-series SE
## beta0        -0.2328  0.2851 0.003681      0.03369
## beta1        -0.7992  0.4155 0.005364      0.04327
## beta2         0.8156  0.3186 0.004112      0.02524
## alpha0       -1.5684  0.3836 0.004952      0.10263
## alpha1        1.0592  0.1637 0.002113      0.02087
## totalAbundance 92.4773 22.6984 0.293035      5.82202
##
## 2. Quantiles for each variable:
##
##              2.5%    25%    50%    75%    97.5%
## beta0        -0.7760 -0.4277 -0.2497 -0.04262  0.366734
## beta1        -1.6647 -1.0528 -0.7880 -0.52895 -0.001139
## beta2         0.2006  0.6004  0.8134  1.02425  1.463198
## alpha0       -2.2776 -1.8919 -1.5084 -1.29019 -0.895715
## alpha1        0.7402  0.9532  1.0498  1.16248  1.397912
## totalAbundance 62.0000 76.0000 86.0000 105.00000 146.000000
```

```
plot(jags.post.samples[,paste0("N[", 1:2, "]")])
```

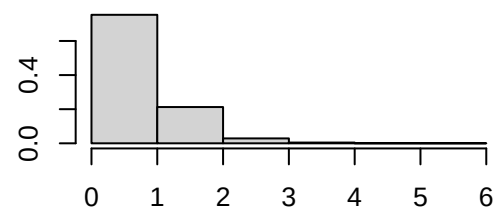


```
plot(jags.post.samples[,paste0("N[", 3:4, "]")])
```

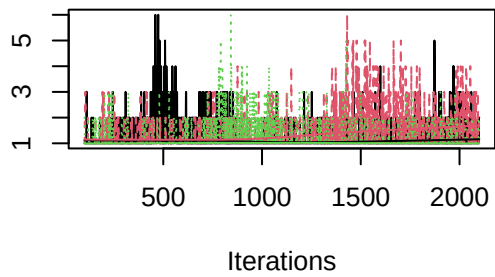
Trace of N[3]



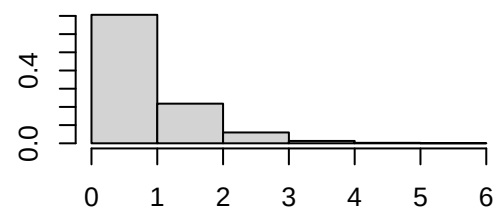
Density of N[3]



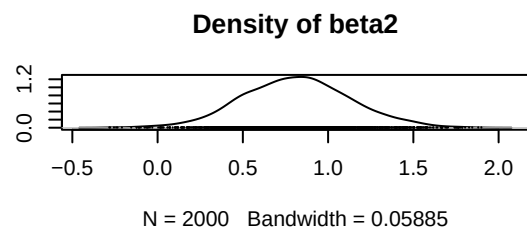
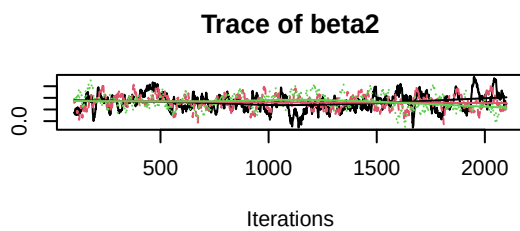
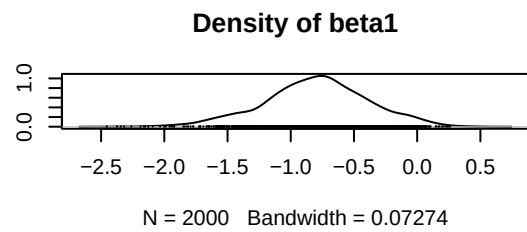
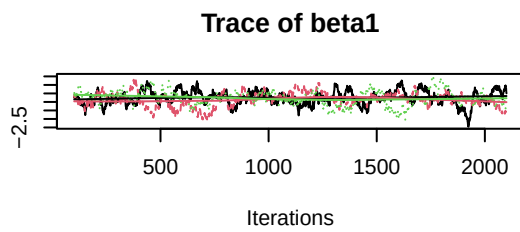
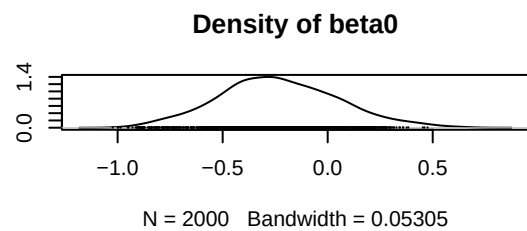
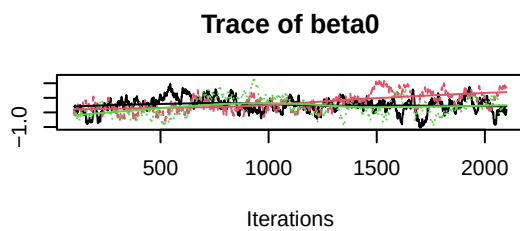
Trace of N[4]



Density of N[4]

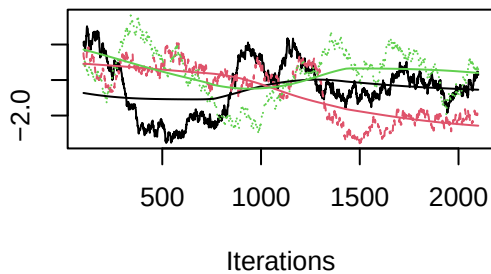


```
plot(jags.post.samples[,jags.pars[1:3]])
```

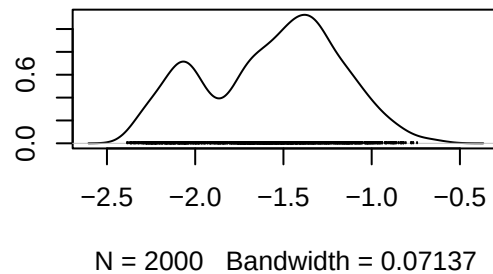


```
plot(jags.post.samples[,jags.pars[4:5]])
```

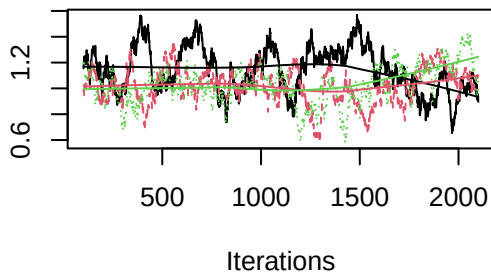
Trace of alpha0



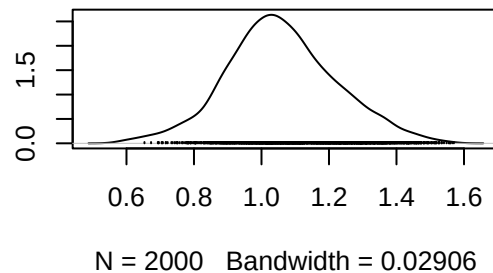
Density of alpha0



Trace of alpha1



Density of alpha1



```
# extract beta values
lambda.coef.post <- as.matrix(jags.post.samples[,c("beta0", "beta1", "beta2")])
head(lambda.coef.post, n=4)
```

```
##           beta0      beta1      beta2
## [1,] -0.4837185 -0.7557389 0.3601422
## [2,] -0.4710857 -0.6743193 0.3341998
## [3,] -0.3907293 -0.7037765 0.3400097
## [4,] -0.3746024 -0.7438384 0.4104394
```

```
n.iter <- nrow(lambda.coef.post)
# vector for predictive val
forest.pred <- c(rep("Hardwood", 50),
                 rep("Mixed", 50),
                 rep("Pine", 50)
                )

forest.Mixed.pred <- ifelse(forest.pred=="Mixed", 1, 0)
forest.Pine.pred <- ifelse(forest.pred=="Pine", 1, 0)

lambda.post.pred <- matrix(NA, nrow=n.iter, ncol=length(forest.pred))

# we now use exp() rather than plogis() since lambda is log-linear rather than logit-linear
for(i in 1:n.iter) {
  lambda.post.pred[i,] <- exp(
```

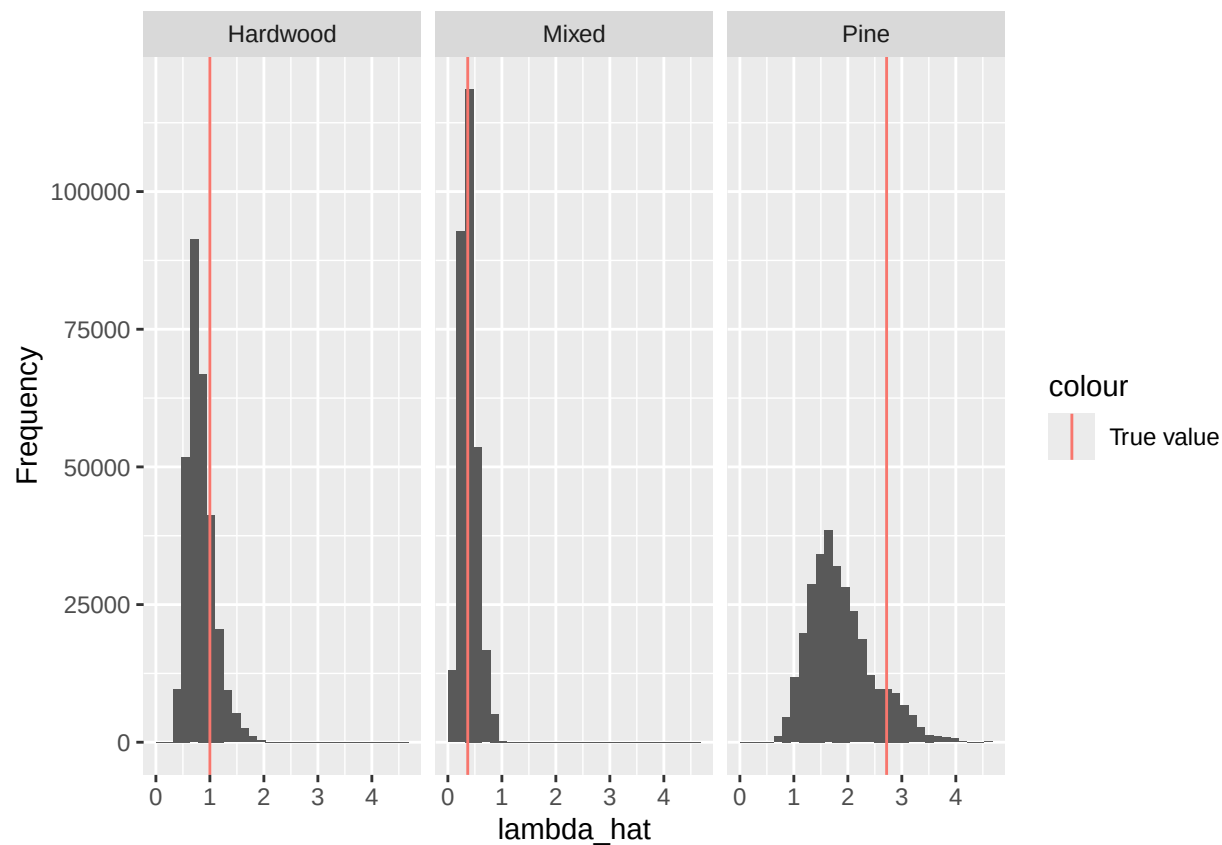
```

    lambda.coef.post[i,"beta0"] +
    lambda.coef.post[i,"beta1"] * forest.Mixed.pred +
    lambda.coef.post[i,"beta2"] * forest.Pine.pred
  )
}

# plot showing forest type on categorical horiz. axis and lambda on continuous vert. vertical axis
# pull out values for each forest type
hardwood <- as.vector(lambda.post.pred[, 1:50])
mixed    <- as.vector(lambda.post.pred[, 51:100])
pine     <- as.vector(lambda.post.pred[, 101:150])
#combine vectors in long form for easy graphing
lambda.long <- data.frame(
  lambda = c(hardwood, mixed, pine),
  forest = rep(
    c("Hardwood", "Mixed", "Pine"),
    times = c(length(hardwood), length(mixed), length(pine))
  )
)
library(ggplot2)
ggplot(lambda.long, aes(x = lambda)) +
  geom_histogram() +
  facet_wrap(~forest, ncol = 3) +
  labs(
    x = "lambda_hat",
    y = "Frequency"
  ) +
  geom_vline(
    data = data.frame(
      forest = names(lambda.true),
      lambda = lambda.true
    ),
    aes(xintercept = lambda, color = "True value"),
    linetype = "solid"
  )
)

```

'stat_bin()' using 'bins = 30'. Pick better value 'binwidth'.



```
jags.data2 <- jags.data # redefine data
jags.data2$y[] <- NA    # Replace data with missing values

# resample
jags.prior.samples <- jags.basic(data=jags.data2,
                                inits=jags.inits,
                                parameters.to.save=jags.pars,
                                model.file="Nmix-model-covs.jag",
                                n.chains=3,
                                n.adapt=100,
                                n.burnin=0,
                                n.iter=2000,
                                parallel=TRUE)
```

```
##
## Processing function input.....
##
## Done.
##
## Beginning parallel processing using 3 cores. Console output will be suppressed.
##
## Parallel processing completed.
##
## MCMC took 0.009 minutes.
```

```
summary(jags.prior.samples)
```

```
##
## Iterations = 1:2000
## Thinning interval = 1
## Number of chains = 3
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##              Mean          SD Naive SE Time-series SE
## alpha0      -0.025196    1.410  0.01820    0.01787
## alpha1      -0.004061    1.414  0.01825    0.01831
## beta0       -0.026819    1.408  0.01818    0.01813
## beta1       -0.003649    1.446  0.01867    0.01867
## beta2       -0.016077    1.405  0.01814    0.01793
## totalAbundance 571.981167 3195.146 41.24916    41.25492
##
## 2. Quantiles for each variable:
##
##              2.5%      25%          50%      75%      97.5%
## alpha0      -2.737 -1.0073  -0.033040   0.9392   2.735
## alpha1      -2.797 -0.9486  -0.006438   0.9323   2.745
## beta0       -2.763 -0.9846  -0.039076   0.9057   2.829
## beta1       -2.796 -1.0054   0.017855   0.9772   2.842
## beta2       -2.835 -0.9629  -0.028170   0.9386   2.702
## totalAbundance 6.000 46.0000 133.000000 393.2500 3465.175
```

```
lambda.coef.prior <- as.matrix(jags.prior.samples[,c("beta0", "beta1", "beta2")])
lambda.prior.pred <- matrix(NA, nrow=n.iter, ncol=length(forest.pred))
for(i in 1:n.iter) {
  lambda.prior.pred[i,] <- exp(lambda.coef.prior[i, "beta0"] + lambda.coef.prior[i, "beta1"]*forest.Mix
}
```

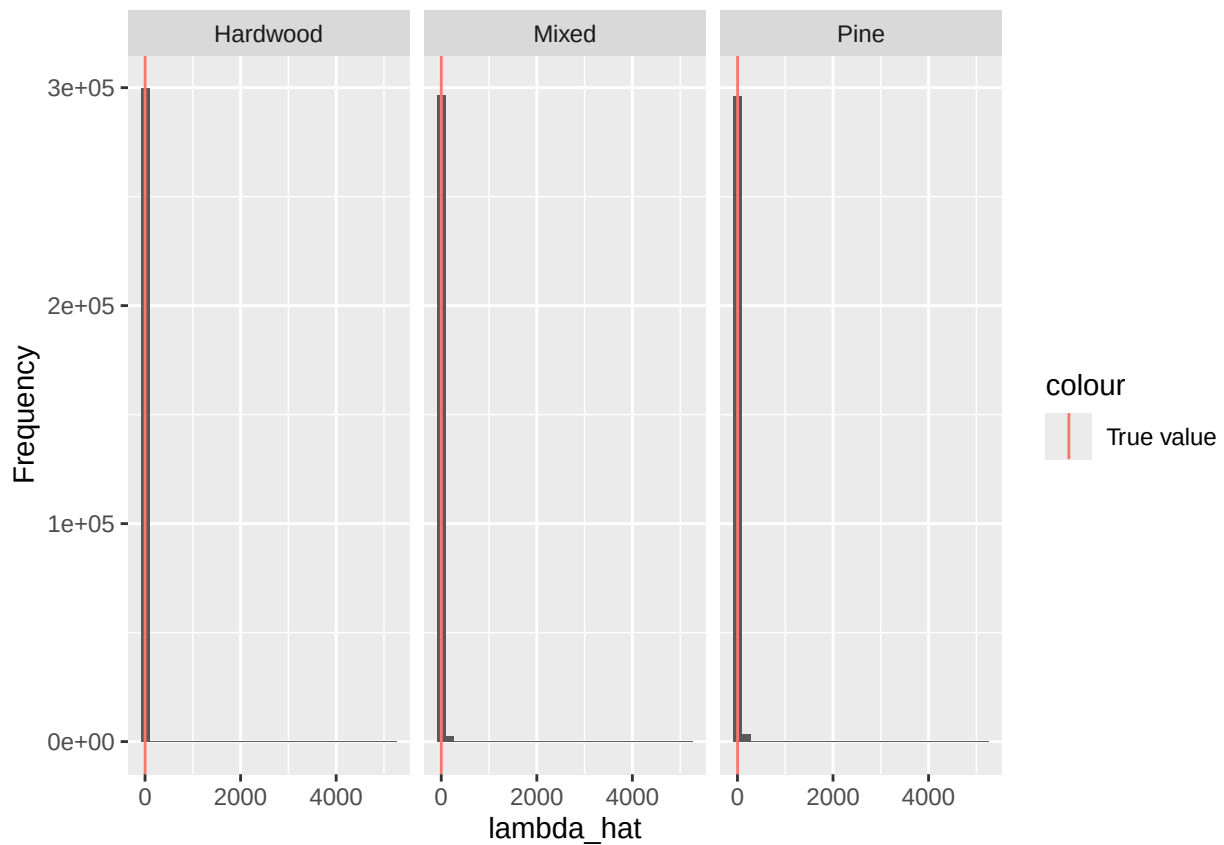
```
# plot showing forest type on categorical horiz. axis and lambda on continuous vert. vertical axis
hardwood.prior <- as.vector(lambda.prior.pred[, 1:50])
mixed.prior <- as.vector(lambda.prior.pred[, 51:100])
pine.prior <- as.vector(lambda.prior.pred[, 101:150])
lambda.long.prior <- data.frame(
  lambda = c(hardwood.prior, mixed.prior, pine.prior),
  forest = rep(
    c("Hardwood", "Mixed", "Pine"),
    times = c(
      length(hardwood.prior),
      length(mixed.prior),
      length(pine.prior)
    )
  )
)
ggplot(lambda.long.prior, aes(x = lambda)) +
  geom_histogram() +
  facet_wrap(~forest, ncol = 3) +
```

```

labs(
  x = "lambda_hat",
  y = "Frequency"
) +
geom_vline(
  data = data.frame(
    forest = names(lambda.true),
    lambda = lambda.true
  ),
  aes(xintercept = lambda, color = "True value"),
  linetype = "solid"
)

```

'stat_bin()' using 'bins = 30'. Pick better value 'binwidth'.



```

lambda.long$distribution <- "Posterior"
lambda.long.prior$distribution <- "Prior"
lambda.combined <- rbind(lambda.long, lambda.long.prior)

ggplot(lambda.combined,
  aes(x = lambda, fill = distribution)) +
  geom_histogram(
    position = "identity",
    alpha = 0.5,

```

```

) +
facet_wrap(~ forest, ncol = 3) +
labs(
  x = "lambda_hat",
  y = "Frequency",
  fill = "Distribution"
)

```

'stat_bin()' using 'bins = 30'. Pick better value 'binwidth'.

